

# Guia de Configuração do Ambiente

Segurança de Sistemas Informáticos, 2025-26

Licenciatura em Engenharia Informática, Universidade do Minho

Vítor Francisco Fonte

2026-04-18

Este guia é a configuração do ambiente para o **Laboratório de Vulnerabilidades de Segurança de Memória (Semana 11)**, que requer uma *shell* Linux x86-64.

---

**Comece aqui:** abra o ficheiro `Lab - Assistente de Configuracao.html` no seu *browser*. Ele deteta o seu sistema operativo e a família do CPU e apresenta-lhe os comandos exactos para o seu caminho de configuração. Volte a este guia apenas se precisar do percurso detalhado de um caminho específico, quiser compreender o raciocínio por detrás de uma escolha, ou estiver a resolver um problema que o assistente não cobriu.

---

## Índice

- [Secção 1. O que vai obter](#)
- [Secção 2. Escolha o seu caminho](#)
  - [2.1 Atalho para veteranos do trabalho de Segurança Web](#)
  - [2.2 Identifique a família do seu CPU](#)
  - [2.3 Escolha o seu caminho](#)
- [Caminho 1. Ubuntu 22.04 x86-64 já existente](#)
- [Caminho 2. Windows com WSL2](#)
- [Caminho 2b. Windows com Docker Desktop](#)
  - [2b.1 Instalar Docker Desktop](#)
  - [2b.2 Criar o container de laboratório](#)
  - [2b.3 Fluxo de trabalho diário](#)
- [Caminho 3. Host x86-64 com VirtualBox](#)
  - [3.1 Instalar o VirtualBox](#)
  - [3.2 Descarregar a ISO do Ubuntu Server](#)
  - [3.3 Criar e configurar a VM](#)
  - [3.4 Instalar o Ubuntu Server](#)
  - [3.5 Ligar a partir da shell do host](#)

- [3.6 Variante Ubuntu Desktop \(opcional\)](#)
  - [Caminho 3b. Mac Intel com Docker Desktop](#)
    - [3b.1 Instalar Docker Desktop e Homebrew](#)
    - [3b.2 Criar o container de laboratório](#)
    - [3b.3 Fluxo de trabalho diário](#)
  - [Caminho 4. Mac Apple Silicon com Docker Desktop e Rosetta](#)
    - [4.1 Instalar Homebrew](#)
    - [4.2 Instalar Docker Desktop](#)
    - [4.3 Ativar a emulação Rosetta x86](#)
    - [4.4 Criar o container de laboratório](#)
    - [4.5 Limitações conhecidas deste caminho](#)
  - [Caminho 5. Mac Apple Silicon com UTM \(VM x86-64 completa\)](#)
  - [Secção 8. Verificar a configuração](#)
  - [Secção 9. Resolução de problemas](#)
  - [Secção 10. Resolução de problemas alargada](#)
    - [10.1 SSH com ligação recusada](#)
    - [10.2 GDB em modo interativo falha dentro do container](#)
    - [10.3 Problemas de teclado](#)
    - [10.4 Confusão com a ordem das colunas de port forwarding no UTM](#)
    - [10.5 Instalação de UTM muito lenta: porque demora tanto e o que fazer](#)
    - [10.6 Erros de compilação: gcc não encontrado, python3 em falta](#)
    - [10.7 A mensagem do stack canary parece uma falha do sistema](#)
    - [10.8 "Normal programme termination" antes de ACCESS GRANTED](#)
- 

## Secção 1. O que vai obter

No final deste guia terá uma *prompt* de *shell* dentro de um ambiente **GNU/Linux x86-64** (Ubuntu 22.04), a partir do qual pode compilar código C e executar scripts Python 3. Todos os exercícios da Semana 11 assumem que tem uma *shell* dessas aberta.

Os enunciados individuais pedem apenas que instale as ferramentas específicas do trabalho (por exemplo `gcc`, `gdb`, `python3` para a Semana 11) em cima dessa *shell*. Não repetem a configuração do sistema operativo descrita abaixo.

---

## Secção 2. Escolha o seu caminho

**Prefere um arranque assistido?** O ficheiro complementar `Lab - Assistente de Configuracao.html` é um assistente numa única página que pode abrir em qualquer *browser*. Deteta automaticamente o seu sistema operativo e a família do CPU, e depois

mostra os comandos exactos para o seu caminho, incluindo os ficheiros *compose* e o fluxo de SSH. Este guia continua a ser a referência autoritativa para os percursos detalhados, as ressalvas e a resolução de problemas; use o assistente para avançar depressa e volte aqui sempre que precisar do contexto.

## 2.1 Atalho para veteranos do trabalho de Segurança Web

Se já realizou o trabalho de Segurança Web na mesma máquina que vai usar agora, já tem o Docker Desktop instalado e conhece o fluxo `docker compose`. Para os caminhos baseados em Docker pode saltar os passos de instalação do Homebrew e do Docker Desktop e ir diretamente para as sub-secções indicadas abaixo.

| A sua máquina                                        | Atalho                                                                                                                                                                    |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| macOS com CPU Intel                                  | Ir diretamente para <a href="#">Caminho 3b, Secção 3b.2</a> (saltar 3b.1).                                                                                                |
| macOS Apple Silicon, a optar pelo Docker (Caminho 4) | <a href="#">Caminho 4, Secção 4.3</a> para ativar o Rosetta (novidade para si), depois <a href="#">Secção 4.4</a> para o novo ficheiro <i>compose</i> . Saltar 4.1 e 4.2. |
| Windows 10/11 (x86-64)                               | Diretamente para <a href="#">Caminho 2b, Secção 2b.2</a> (saltar 2b.1).                                                                                                   |

O ficheiro *compose* usado para o trabalho de Segurança de Software **não** é o do trabalho de Segurança Web. O trabalho de Segurança Web executava um ambiente SEED Labs com múltiplos *containers*; aqui executa um único *container* Ubuntu como bancada de trabalho, com um *bind-mount* para uma pasta no seu *host*. Use o ficheiro *compose* indicado na Secção 2b.2, Secção 3b.2 ou Secção 4.4, não o que guardou da anterior.

**Nota para alunos em Apple Silicon.** O caminho *recomendado* para este trabalho é o Caminho 5 (UTM), porque o Exercício 2 da Semana 11 é construído em torno de uma sessão interativa de GDB e o Rosetta sob Docker não suporta `ptrace` sobre processos traduzidos. O Caminho 4 é oferecido como opção para alunos que prefiram o fluxo Docker mais leve e que aceitem seguir a alternativa de análise estática ao Exercício 2 documentada no enunciado. Se escolher o Caminho 5, o atalho acima não se aplica; siga o percurso completo do Caminho 5.

Se preferir uma abordagem diferente, ou se o seu *hardware* não for Mac Intel nem Apple Silicon, siga a selecção completa de caminhos abaixo.

## 2.2 Identifique a família do seu CPU

Execute a verificação adequada ao seu sistema operativo:

- **macOS ou Linux:** num terminal, execute `uname -m`.
- **Windows:** abra *Definições, Sistema, Sobre* e leia a linha **Tipo de sistema**.

Depois interprete o resultado:

- Se vir `x86_64` (macOS/Linux) ou "processador baseado em x64" (Windows), tem uma máquina **x86-64** (Intel ou AMD). Qualquer um dos Caminhos 1 a 3 e o Caminho 3b funcionará.
- Se vir `arm64` ou `aarch64` (macOS/Linux) ou "processador baseado em ARM" (Windows), tem uma máquina **ARM**. Um ambiente x86-64 só pode ser obtido por emulação. Em macOS Apple Silicon, o Caminho 5 é o recomendado por defeito (preserva o GDB em modo interativo); o Caminho 4 é uma alternativa mais leve. Em outras plataformas ARM, veja a última linha da tabela abaixo.

## 2.3 Escolha o seu caminho

| A sua situação                                                                  | Caminho recomendado                                                              | Notas                                                                                                                          |
|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Ubuntu 22.04 x86-64 já a executar (instalação nativa, distribuição WSL2 ou VM)  | <a href="#">Caminho 1</a>                                                        | Nada a instalar; basta verificar.                                                                                              |
| Windows 10/11 x86-64, já utilizou Docker no trabalho de Segurança Web           | <a href="#">Caminho 2b</a>                                                       | Docker Desktop com <i>container</i> Ubuntu x86-64. Suporte completo de GDB. Reutiliza a instalação existente.                  |
| Windows 10/11 x86-64, ainda sem Docker                                          | <a href="#">Caminho 2</a> (mais leve) ou <a href="#">Caminho 3</a> (VM completa) | O WSL2 tem a menor pegada. O VirtualBox dá-lhe um <i>kernel</i> completo e facilita o uso de GDB.                              |
| macOS x86-64 (Intel), a querer uma VM completa                                  | <a href="#">Caminho 3</a>                                                        | VirtualBox com Ubuntu Server x86-64 como <i>guest</i> .                                                                        |
| macOS x86-64 (Intel), já utilizou Docker no trabalho de Segurança Web           | <a href="#">Caminho 3b</a>                                                       | Docker Desktop com <i>container</i> Ubuntu x86-64. Execução nativa, suporte completo de GDB.                                   |
| Linux x86-64 (não-Ubuntu, ou a querer uma VM isolada)                           | <a href="#">Caminho 3</a>                                                        | VirtualBox com Ubuntu Server x86-64 como <i>guest</i> .                                                                        |
| <b>macOS em Apple Silicon</b> (M1/M2/M3/M4), recomendado                        | <a href="#">Caminho 5</a>                                                        | VM x86-64 em UTM. Preserva GDB interativo; só é lento nos eventos de configuração únicos.                                      |
| macOS em Apple Silicon, opção mais leve                                         | <a href="#">Caminho 4</a>                                                        | Docker Desktop com Rosetta. Menor pegada, mas GDB interativo não funciona; use a alternativa de análise estática do enunciado. |
| Qualquer máquina ARM (Raspberry Pi, Asahi Linux, Windows ARM como Snapdragon X) | <a href="#">Caminho 4</a> (o Caminho 5 é exclusivo de macOS)                     | Siga as instruções de Docker para Apple Silicon; x86-64 terá de ser emulado.                                                   |

**Porque não há uma única recomendação?** A virtualização x86-64 requer um CPU x86-64. Em *hardware* x86-64 (Intel ou AMD), qualquer *hypervisor* (VirtualBox, Docker Desktop em Mac Intel, WSL2, Hyper-V) executa Linux x86-64 a velocidade nativa, pelo que a escolha é sobretudo uma questão de fluxo de trabalho e pegada em disco. Em *hardware* ARM, o programa tem de ser *traduzido* para arm64 em tempo de execução. Para o trabalho de Segurança de Software recomendamos o caminho que preserva o **GDB em modo interativo**: em Apple Silicon, esse é o Caminho 5 (UTM), porque o Exercício 2 da Semana 11 assenta numa sessão interativa de *debug* e o Rosetta sob Docker não consegue fazer `ptrace` dos processos que traduz. O Caminho 4 (Docker com Rosetta) está disponível como opção mais leve, com uma alternativa de análise estática ao Exercício 2. Em *hardware* ARM fora de Apple, o UTM não está disponível, pelo que o Caminho 4 é a única opção.

---

## Caminho 1. Ubuntu 22.04 x86-64 já existente

Use este caminho se já tem um ambiente Ubuntu 22.04 onde pode abrir uma *shell*, quer se trate de:

- Uma instalação Ubuntu nativa,
- Uma distribuição Ubuntu em WSL2,
- Ou uma VM pré-existente (VirtualBox, VMware, Hyper-V, Parallels, UTM, etc.).

**Pré-requisito:** o ambiente tem de ser x86-64. Abra uma *shell* nele e execute:

```
uname -m
```

Se o resultado for `x86_64`, está pronto ao nível do SO. Avance para [Secção 8: Verificar a configuração](#). As ferramentas específicas do trabalho são instaladas por cada enunciado com `sudo apt install ...`.

Se o resultado for `aarch64` ou outro, o seu Ubuntu é ARM (comum em Raspberry Pi, Asahi Linux, ou uma VM ARM). Este trabalho requer x86-64, pelo que deve usar o Caminho 4 ou o Caminho 5.

---

## Caminho 2. Windows com WSL2

**Pré-requisito:** uma máquina Windows **x86-64**. Verifique em *Definições, Sistema, Sobre*: a linha *Tipo de sistema* deve indicar "processador baseado em x64". Numa máquina Windows baseada em ARM (por exemplo um Surface Pro com *chip* Snapdragon X, ou qualquer outro PC ARM64), o WSL2 instala por omissão um Ubuntu **arm64** e distribuições x86-64 teriam

de ser executadas sob emulação. Trate tal máquina como Apple Silicon e use o Caminho 4 ou o Caminho 5.

1. Abra o **PowerShell como Administrador** e execute:

```
wsl --install -d Ubuntu-22.04
```

2. Reinicie quando solicitado. No próximo arranque, um terminal Ubuntu abre-se automaticamente e pede um nome de utilizador e palavra-passe Linux. Escolha algo que consiga lembrar-se; é independente da sua conta Windows.
3. A partir daí, abra o Ubuntu a partir do menu Iniciar (ou execute `wsl` em qualquer terminal).
4. Confirme que está num Linux x86-64:

```
uname -m          # deve imprimir x86_64
lsb_release -d     # deve indicar Ubuntu 22.04
```

Avance para [Secção 8: Verificar a configuração](#).

**Multipass** é uma alternativa aceitável ao WSL2 e funciona de forma equivalente para efeitos desta disciplina. Se já o usa, não há nada a mudar.

## Caminho 2b. Windows com Docker Desktop

Se já completou o trabalho de Segurança Web nesta máquina Windows, o Docker Desktop já está instalado e conhece o fluxo `docker compose`. Este caminho reutiliza essa configuração com um ficheiro *compose* diferente e é o ponto de partida preferido para utilizadores Windows com experiência Docker prévia.

No Windows, o Docker Desktop utiliza o WSL2 como *kernel* Linux de *backend*: o *container* corre num *kernel* Linux x86-64 genuíno, não em instruções emuladas. A opção `cap_add: SYS_PTRACE` funciona, portanto, como pretendido, e o GDB em modo interativo funciona exatamente como descrito no enunciado. A limitação do Rosetta documentada no Caminho 4 não se aplica aqui.

**Pré-requisito:** uma máquina Windows **x86-64**. Verifique em *Definições, Sistema, Sobre*: a linha *Tipo de sistema* deve indicar "processador baseado em x64". Numa máquina Windows baseada em ARM, siga o Caminho 4 (ver a última linha da tabela da Secção 2.3).

### 2b.1 Instalar Docker Desktop (saltar se já tiver)

- **Se já completou o trabalho de Segurança Web nesta máquina:** o Docker Desktop e o WSL2 já estão configurados. Avance para a Secção 2b.2.
- **Caso contrário:** descarregue o **Docker Desktop for Windows** em <https://www.docker.com/products/docker-desktop/> e execute o *installer* com as opções

por defeito. Instala e configura o WSL2 automaticamente como *backend*. Reinicie quando solicitado e depois lance o Docker Desktop a partir do menu Iniciar; aguarde até o ícone da baleia na área de notificação da barra de tarefas parar de animar.

## 2b.2 Criar o container de laboratório

Abra o **Windows Terminal** (ou PowerShell) e escolha uma pasta para o seu trabalho:

```
mkdir ~/ssi-lab/work  
cd ~/ssi-lab
```

Crie um `docker-compose.yml` dentro de `~/ssi-lab` com exatamente este conteúdo:

```
services:  
  lab:  
    image: ubuntu:22.04  
    platform: linux/amd64  
    volumes:  
      - ./work:/root/work  
    working_dir: /root/work  
    command: sleep infinity  
    stdin_open: true  
    tty: true  
    # Required so GDB can ptrace target programs inside the container.  
    cap_add:  
      - SYS_PTRACE  
    security_opt:  
      - seccomp=unconfined
```

**Nota para veteranos do trabalho de Segurança Web.** Este ficheiro *compose* não é o que usou nesse trabalho. A configuração do trabalho de Segurança Web executava um ambiente SEED Labs com múltiplos *containers*; aqui executa um único *container* Ubuntu como bancada de trabalho, com um *bind-mount* para uma pasta no seu *host* Windows. A linha `platform: linux/amd64` é redundante em Windows x86-64 (a predefinição já é amd64) mas inofensiva, e mantém este ficheiro intercambiável com os caminhos Docker do Mac.

Lance o *container* e abra uma *shell*:

```
docker compose up -d  
docker compose exec lab bash
```

A sua *prompt* muda para algo como `root@<id>:/root/work#`. Os ficheiros que criar em `work/` no seu *host* Windows aparecem imediatamente em `/root/work` dentro do *container*; use essa pasta para todos os ficheiros de *source* do trabalho. Confirme:



```
uname -m
```

```
# x86_64
```

## 2b.3 Fluxo de trabalho diário

`docker compose up -d` lança o *container*, `docker compose exec lab bash` abre uma *shell*, `exit` sai da *shell* (o *container* continua a correr), `docker compose down` pára e remove.

Avance para [Secção 8: Verificar a configuração](#).

---

## Caminho 3. Host x86-64 com VirtualBox

Este caminho dá-lhe um *kernel* Linux x86-64 **real**, pelo que todos os exercícios da Semana 11 (incluindo GDB interativo) se comportam exatamente como o enunciado descreve. Funciona de modo idêntico num Mac Intel, num Windows 10/11 com Intel/AMD ou num *host* Linux x86-64; apenas o passo de instalação do VirtualBox difere e a *shell* do *host* usada para fazer SSH para a VM varia (Terminal do macOS, Windows Terminal ou PowerShell no Windows, ou o terminal Linux da sua preferência).

O *guest* recomendado é o **Ubuntu 22.04 Server** (instalação sem ambiente gráfico). É mais pequeno, mais rápido de instalar, e irá interagir com ele através de SSH a partir da *shell* do *host*, o que lhe dá *copy/paste* nativo e uma experiência de edição muito mais confortável do que a janela de consola do VirtualBox. Se preferir um ambiente gráfico, uma variante Desktop é descrita no final desta secção.

### 3.1 Instalar o VirtualBox

Escolha a linha que corresponde ao seu *host*:

- **macOS (Intel):** ou descarregue o *installer* macOS/Intel em <https://www.virtualbox.org/wiki/Downloads>, ou execute `brew install --cask virtualbox`. No primeiro arranque, o macOS pedirá permissão para carregar uma extensão de sistema; aprobe em **Definições do Sistema, Privacidade e Segurança** e reinicie o Mac se solicitado.
- **Windows 10/11 (x86-64):** descarregue o *installer* "Windows hosts" em <https://www.virtualbox.org/wiki/Downloads> e execute-o com as opções por defeito. Se o Windows reclamar que o Hyper-V está ativo, ou desative o Hyper-V ou aceite o *backend* "Microsoft Hypervisor Platform" de desempenho reduzido que o VirtualBox usará automaticamente.
- **Linux (x86-64):** instale a partir da sua distribuição ( `sudo apt install virtualbox` em Debian/Ubuntu) ou siga o repositório apt oficial da Oracle em [https://www.virtualbox.org/wiki/Linux\\_Downloads](https://www.virtualbox.org/wiki/Linux_Downloads) para a versão mais recente.



Verifique lançando a aplicação VirtualBox; deverá ver a janela do VirtualBox Manager.

## 3.2 Descarregar a ISO do Ubuntu Server

Descarregue o **Ubuntu 22.04.5 LTS Server** (amd64, NÃO arm64) em <https://releases.ubuntu.com/22.04/>. O ficheiro tem cerca de 2 GB.

## 3.3 Criar e configurar a VM

No VirtualBox, clique em *Novo* e preencha:

- Nome: `ssi-lab`
- Tipo: Linux, Versão: Ubuntu (64-bit)
- Memória: 2 GB (mais não faz mal)
- Crie um disco virtual de 15 GB (VDI, dinamicamente alocado).

**Pegada em disco.** O valor de 15 GB é o *teto virtual*, não o que a VM irá ocupar no SSD do seu *host*. Num VDI dinamicamente alocado, o ficheiro em disco cresce apenas à medida que o *guest* escreve nele. Uma instalação de Ubuntu Server com as ferramentas do trabalho fica em cerca de **5 a 7 GB** de uso real; a ISO de 2 GB pode ser apagada após a instalação. A variante Desktop duplica aproximadamente essa pegada.

Depois de criada a VM, clique em *Definições* e faça três ajustes:

1. *Armazenamento*: clique na *drive* ótica vazia e anexe a ISO do Ubuntu Server que descarregou.
2. *Rede, Adaptador 1*: deixe *Ligado a* como **NAT**. Clique em *Avançado, Redirecionamento de Portas* e adicione uma regra:

| Nome | Protocolo | IP do Host | Porta do Host | IP do Guest | Porta do Guest |
|------|-----------|------------|---------------|-------------|----------------|
| SSH  | TCP       | (vazio)    | 2222          | (vazio)     | 22             |

Isto permite-lhe aceder ao servidor SSH da VM a partir do seu *host* em `localhost:2222`.

3. *Sistema, Processador*: defina 2 CPUs se o seu *host* tiver quatro ou mais núcleos.

## 3.4 Instalar o Ubuntu Server

Inicie a VM. O *installer* é em modo texto e navega-se com as teclas de setas e Tab. Aceite o idioma por omissão. No ecrã de **Configuração do Teclado**, defina **Layout: Portuguese** e **Variant: Portuguese - Portuguese (Macintosh)** se (como será o caso da maior parte dos alunos desta disciplina) estiver a usar um MacBook com teclado português; caso contrário, escolha o *layout* e a variante correspondentes ao seu teclado físico. Depois, no ecrã **Configuração do perfil**, escolha um nome de utilizador e palavra-passe que consiga memorizar.

No ecrã **Configuração SSH**, marque **Install OpenSSH server**. Esta é a escolha mais importante deste caminho: é o que lhe permite dispensar a janela de consola da VM para o resto do trabalho. Não precisa de importar uma chave SSH a menos que já use uma.

No ecrã **Featured Server Snaps**, não selecione nada; carregue em *Done*.

Quando o *installer* terminar e pedir para reiniciar, desligue a VM (menu do VirtualBox: *Máquina, Encerramento ACPI*). Abra *Definições, Armazenamento* na VM, retire a ISO, e arranque a VM de novo.

### 3.5 Ligar a partir da *shell* do seu *host*

Com a VM a correr (deverá ver uma *prompt* de login na janela do VirtualBox, mas já não precisa de escrever lá), abra uma *shell* no seu *host* e execute:

- **macOS ou Linux:** use o Terminal (macOS) ou qualquer emulador de terminal (Linux). O comando `ssh` é nativo.
- **Windows 10/11:** use o Windows Terminal ou PowerShell. O comando `ssh` faz parte do *OpenSSH Client*, uma funcionalidade opcional que está activa por defeito em Windows 10 (versão 1809 ou superior) e Windows 11. Se o `ssh` não for reconhecido, instale-o em *Definições, Aplicações, Funcionalidades opcionais, Adicionar uma funcionalidade, OpenSSH Client*.

Depois ligue-se:

```
ssh -p 2222 <o-seu-utilizador>@localhost
```

Aceite a *host key fingerprint* na primeira ligação, introduza a palavra-passe escolhida durante a instalação, e está agora numa sessão confortável com *copy/paste* nativo a partir do sistema operativo do *host*. Confirme:

```
uname -m          # x86_64
lsb_release -d     # Ubuntu 22.04
```

Avance para [Secção 8: Verificar a configuração](#). Daqui em diante, trate a VM exatamente como no Caminho 1: cada `sudo apt install ...` num enunciado é executado nesta sessão SSH.

**Dica.** Para evitar escrever o comando `ssh` completo de cada vez, adicione um bloco ao seu ficheiro de configuração SSH (`~/.ssh/config` em macOS/Linux; `%USERPROFILE%\ssh\config` em Windows):

```
Host ssi-lab
  HostName localhost
```

```
Port 2222
User <o-seu-utilizador>
```

Depois disto, `ssh ssi-lab` basta. Para copiar ficheiros do *host* para a VM use `scp -P 2222 ficheiro.txt ssi-lab:~`, ou simplesmente `scp ficheiro.txt ssi-lab:~` quando a configuração estiver no lugar.

### 3.6 Variante Ubuntu Desktop (opcional)

Se quiser especificamente um ambiente gráfico dentro da VM, siga os passos acima com estas substituições:

- Descarregue o **Ubuntu 22.04.5 LTS Desktop** em vez do Server (cerca de 4,5 GB).
- Dê à VM 4 GB de memória e um disco de 25 GB.
- Durante a instalação escolha uma instalação *Normal*; o resto é idêntico ao Server.
- Não precisa estritamente da regra de *port-forward*, porque pode usar o Terminal incorporado da VM (Ctrl+Alt+T) e o Firefox dentro da VM. A instalação e activação do OpenSSH continua a ser recomendada para manter a opção de SSH a partir do seu *host*:

```
sudo apt install openssh-server
sudo systemctl enable --now ssh
```

- Para partilha de *clipboard* entre o SO do *host* e a VM Desktop, instale as Guest Additions dentro da VM (`sudo apt install virtualbox-guest-utils`) e active *Dispositivos*, *Área de Transferência Partilhada*, *Bidireccional* no menu do VirtualBox.

O resto do trabalho é idêntico independentemente da variante Ubuntu instalada.

---

## Caminho 3b. Mac Intel com Docker Desktop

Num Mac Intel, o Docker Desktop executa uma VM Linux x86-64 de forma *nativa*, sem camada de tradução. *Containers* criados para `linux/amd64` executam portanto à velocidade nativa e todas as funcionalidades de depuração (*breakpoints* de GDB, `info frame`, *stepping*, aleatorização ASLR) funcionam exatamente como num *host* Linux nativo. As ressalvas associadas ao Rosetta (Caminho 4) não se aplicam a si.

Esta é a opção mais leve para alunos com Mac Intel que já completaram o trabalho de Segurança Web: reaproveita o mesmo fluxo Docker, apenas com um ficheiro *compose* diferente.

### 3b.1 Instalar Docker Desktop e Homebrew

- **Se já completou o trabalho de Segurança Web:** já tem o Docker Desktop e o Homebrew instalados. Salte esta sub-secção e avance para 3b.2.

- **Caso contrário:** siga a [Secção 4.1](#) para instalar o Homebrew e depois a [Secção 4.2](#) para instalar o Docker Desktop. Pare após a Secção 4.2 e volte para aqui. *Não* precisa da Secção 4.3 (Rosetta): num Mac Intel não há nada a traduzir.

## 3b.2 Criar o container de laboratório

Escolha uma pasta para o seu trabalho:

```
mkdir -p ~/ssi-lab/work
cd ~/ssi-lab
```

Crie um `docker-compose.yml` dentro de `~/ssi-lab` com exatamente este conteúdo:

```
services:
  lab:
    image: ubuntu:22.04
    platform: linux/amd64
    volumes:
      - ./work:/root/work
    working_dir: /root/work
    command: sleep infinity
    stdin_open: true
    tty: true
    # Required so GDB can ptrace target programs inside the container.
    cap_add:
      - SYS_PTRACE
    security_opt:
      - seccomp=unconfined
```

**Nota para veteranos do trabalho de Segurança Web.** Este ficheiro *compose* não é o que usou nesse trabalho. A configuração do trabalho de Segurança Web executava um ambiente SEED Labs com múltiplos *containers*, mapeamentos de portas e um IP fixo; aqui executa um único *container* Ubuntu como bancada de trabalho, com um *bind-mount* para uma pasta no seu Mac. A linha `platform: linux/amd64` é redundante em Macs Intel (a predefinição já é amd64) mas inofensiva, e mantém este ficheiro intercambiável com o do Caminho 4.

Lance o *container* e abra uma *shell*:

```
docker compose up -d
docker compose exec lab bash
```

A sua *prompt* muda para algo como `root@<id>:/root/work#`. Ficheiros em `work/` no seu Mac aparecem em `/root/work` dentro do *container*. Confirme:

```
uname -m
```

```
# x86_64
```

### 3b.3 Fluxo de trabalho diário

Idêntico ao Caminho 4: `docker compose up -d` para lançar o *container*, `docker compose exec lab bash` para obter uma *shell*, `exit` para sair da *shell* (o *container* continua a correr), `docker compose down` para parar e remover.

Avance para [Secção 8: Verificar a configuração](#).

## Caminho 4. Mac Apple Silicon com Docker Desktop e Rosetta

**Caminho opcional.** O caminho recomendado para Macs Apple Silicon é o **Caminho 5 (UTM)**, porque o Exercício 2 do trabalho de Segurança de Software assenta numa sessão interativa de GDB que não funciona sob Docker com Rosetta (ver as limitações no final desta secção). Escolha o Caminho 4 apenas se preferir um fluxo Docker mais leve e rápido de instalar e aceitar seguir a alternativa de análise estática ao Exercício 2 fornecida no enunciado.

O trabalho corre dentro de um *container* Ubuntu 22.04 x86-64; o seu Mac continua a correr macOS/arm64 por baixo, e o **Rosetta** da Apple traduz silenciosamente instruções x86-64 para arm64 quase à velocidade nativa.

**Já completou o trabalho de Segurança Web?** Já tem o Homebrew e o Docker Desktop instalados, e conhece o fluxo `docker compose`. Salte as secções 4.1 e 4.2 e vá diretamente para [Secção 4.3](#) para ativar o Rosetta (novidade para si) e depois [Secção 4.4](#) para o novo ficheiro *compose*. O ficheiro *compose* usado aqui não é o do trabalho de Segurança Web; é um *container* Ubuntu único em vez da configuração SEED Labs com múltiplos *containers*.

### 4.1 Instalar Homebrew (caso ainda não tenha)

O Homebrew é o gestor de pacotes padrão do macOS. No Terminal:

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

- Introduza a palavra-passe do macOS quando solicitada (os caracteres não aparecem enquanto escreve).
- Terminada a instalação, siga os dois comandos que aparecem mesmo no fim. Adicionam o Homebrew ao `PATH` da sua *shell*. Em Apple Silicon têm este aspeto:

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

- Verifique: `brew --version` deve imprimir Homebrew 5.x.x.

## 4.2 Instalar Docker Desktop

```
brew install --cask docker-desktop
```

Será solicitada uma única vez a palavra-passe do macOS; o *cask* precisa dela para criar uma diretoria de sistema para os *plugins* de CLI do Docker. Lance o Docker Desktop após a instalação:

```
open -a Docker
```

No primeiro arranque:

- Aceite a licença.
- Autorize a instalação do *helper* privilegiado (pode ser pedida novamente a palavra-passe).
- Pode saltar os tutoriais de *onboarding*.

Espere que o ícone da baleia na barra de menus do macOS pare de animar, depois confirme a partir de um terminal:

```
docker info
```

Deverá ver uma secção `Server` com um número de versão.

## 4.3 Ativar a emulação Rosetta x86

1. Clique no **ícone da baleia** na barra de menus e escolha **Settings...**
2. Na barra lateral, escolha **General**.
3. Defina **Virtual Machine Manager** para **Apple Virtualization framework**.
4. Marque **Use Rosetta for x86\_64/amd64 emulation on Apple Silicon**.
5. Clique em **Apply & restart**. Espere cerca de 30 segundos.

Se o Rosetta não estiver instalado no seu Mac, o macOS pedirá para o instalar; aceite.

## 4.4 Criar o container de laboratório

Escolha uma pasta para o seu trabalho (uma única pasta chega para todos os ficheiros da Semana 11):

```
mkdir -p ~/ssi-lab/work
cd ~/ssi-lab
```

Crie um `docker-compose.yml` dentro de `~/ssi-lab` com exatamente este conteúdo:

```
services:
  lab:
    image: ubuntu:22.04
    platform: linux/amd64
    volumes:
      - ./work:/root/work
    working_dir: /root/work
    command: sleep infinity
    stdin_open: true
    tty: true
    # Required so GDB can ptrace target programs inside the container.
    cap_add:
      - SYS_PTRACE
    security_opt:
      - seccomp=unconfined
```

Lance o *container* e abra uma *shell*:

```
docker compose up -d
docker compose exec lab bash
```

A sua *prompt* muda para algo como `root@<id>:/root/work#`. Está agora dentro do ambiente Ubuntu x86-64. Os ficheiros que criar em `work/` no Mac aparecem imediatamente em `/root/work` dentro do *container*; use essa pasta para todos os ficheiros de *source* do trabalho.

**Fluxo diário.** `docker compose up -d` lança o *container*, `docker compose exec lab bash` abre uma *shell*, `exit` sai da *shell* (o *container* continua a correr), `docker compose down` pára e remove.

## 4.5 Limitações conhecidas deste caminho

- **O GDB em modo interativo (breakpoints, info frame, stepping) não funciona** dentro do *container*. O Rosetta não consegue fazer `ptrace` dos processos que traduz, pelo que o GDB falha com `Couldn't get registers: Input/output error`. Cada trabalho que requer GDB em tempo real inclui uma *alternativa de análise estática* que pode usar no seu lugar.
- **PIE/ASLR do código do programa fica neutralizada** pela *cache* de tradução do Rosetta: o endereço de `secret_function` aparecerá idêntico entre execuções mesmo com ASLR ativa. Os endereços da *stack*, do *heap* e das bibliotecas *continuam* a ser



aleatorizados. Os trabalhos que analisam este comportamento pedirão que descreva o que observa e o que aconteceria num x86-64 nativo.

Se alguma destas limitações for crítica para o resultado de aprendizagem que pretende, use o Caminho 5.

Avance para [Secção 8: Verificar a configuração](#).

---

## Caminho 5. Mac Apple Silicon com UTM (VM x86-64 completa)

**Caminho recomendado para Apple Silicon.** Este caminho executa um *kernel* Linux x86-64 totalmente emulado, pelo que todos os exercícios do trabalho de Segurança de Software (incluindo a sessão de GDB interativa do Exercício 2) se comportam exatamente como em *hardware* nativo. O Caminho 4 (Docker com Rosetta) é uma alternativa mais leve, mas o Rosetta não consegue fazer `ptrace` dos processos que traduz, pelo que o GDB em tempo real não funciona aí.

A VM emula um CPU x86-64 em *software*, pelo que trabalho limitado por CPU corre aproximadamente **5 a 10 vezes mais lentamente** do que no Caminho 4.

Para o trabalho de Segurança de Software, essa lentidão *não* é uma preocupação prática durante os exercícios propriamente ditos: o trabalho compila ficheiros C minúsculos e executa-os sobre entradas curtas, tudo concluído em muito menos de um segundo mesmo a velocidade emulada. Onde *irá* notar a emulação é nos eventos de configuração únicos.

**Reserve 60 a 90 minutos ou mais para a instalação inicial do Ubuntu:** o UTM emula x86-64 em *software* e o Ubuntu 22.04 aplica vários anos de atualizações de segurança acumuladas durante a instalação. O primeiro `apt install gcc gdb python3` dentro do *guest* adiciona mais alguns minutos. Faça esta configuração **antes** da sessão de laboratório, não durante, e veja o aviso no topo do enunciado da Semana 11 para saber o que fazer se chegar à aula sem um ambiente funcional.

**Pegada em disco.** Uma instalação de Ubuntu Server em UTM com disco virtual dinamicamente alocado usa cerca de **5 a 8 GB** no SSD do seu Mac depois de o sistema estar instalado e as ferramentas do trabalho no lugar, mais a ISO de 2 GB até a apagar. Ser-lhe-á pedido que configure um tamanho *virtual* de disco (20 GB é confortável), mas apenas a parte efetivamente escrita é armazenada no Mac. Uma variante Desktop duplica aproximadamente essa pegada.

1. **Instale o UTM e mantenha-o atualizado.** A interface das *Settings*, *Network* do UTM (incluindo o painel de *Port Forwarding* coberto no passo 3) mudou entre versões recentes, pelo que convém usar uma versão atualizada:

```
brew install --cask utm          # primeira vez
brew upgrade --cask utm         # se já tinha instalado antes
```

Se instalou o UTM manualmente a partir de <https://mac.getutm.app/>, verifique actualizações a partir da própria aplicação: *menu UTM, Check for Updates...*. Estas instruções foram validadas em UTM 4.7.5; se vir um *layout* obviamente diferente numa versão muito mais antiga, atualize primeiro.

2. **Descarregue a ISO Ubuntu 22.04 Server amd64** (NÃO arm64) em <https://releases.ubuntu.com/22.04/>. À data de escrita, o ficheiro é `ubuntu-22.04.5-live-server-amd64.iso` (cerca de 2 GB). As ISOs Desktop também funcionam; a Server é mais pequena e arranca mais depressa.
3. **Crie a VM.** Clique no botão **+** no topo da barra lateral do UTM e escolha **Emulate**. O *wizard* do UTM apresenta então estes ecrãs nesta ordem:
  - **Hardware:** Arquitetura `x86_64` (predefinição em Apple Silicon quando emulando), aceite o *System* por omissão, Memória **4096 MB**, *CPU Cores 2*.
  - **Operating System:** escolha **Linux**. Neste ecrã, configure **Boot ISO Image** (selecione a ISO do Ubuntu Server que descarregou).
  - **Storage:** tamanho **20 GB**.
  - **Shared Directory:** salte (deixe em branco).
  - **Summary:** dê um nome à VM (`ssi-lab-serve`). Imediatamente abaixo do campo Name, **marque Open VM Settings** antes de clicar em Save. Isto leva-o directamente para o diálogo de *Settings*, que é onde o próximo passo acontece.
4. **Configure o port forwarding antes do primeiro arranque**, para que possa fazer SSH para o *guest* a partir do Terminal do seu Mac assim que o Ubuntu estiver instalado. No diálogo de *Settings* da VM:
  - Seleccione **Network** na barra lateral esquerda.
  - Altere **Network Mode** de *Shared Network* (omissão) para **Emulated VLAN**. (As outras opções de *Network Mode*, ou seja *Shared Network*, *Host Only* e *Bridged (Advanced)*, não expõem um editor de *port forwarding*.)
  - Uma nova entrada **Port Forwarding** surge na barra lateral esquerda. Seleccione-a.
  - Clique em **New** e preencha (as colunas do UTM surgem nesta ordem):

| Protocol | Guest Address | Guest Port | Host Address | Host Port |
|----------|---------------|------------|--------------|-----------|
| TCP      | (vazio)       | 22         | (vazio)      | 2222      |

- Feche o diálogo de *Settings* para guardar.

Isto mapeia `localhost:2222` no seu Mac para a porta 22 (SSH) dentro da VM, logo que o *guest* esteja ligado e com um servidor SSH à escuta.

5. **Instale o Ubuntu Server.** Inicie a VM a partir da janela principal do UTM; o *installer* arranca a partir da ISO. Aceite o idioma por omissão. No ecrã **Keyboard configuration**,

defina **Layout: Portuguese** e **Variant: Portuguese - Portuguese (Macintosh)** se estiver a usar um MacBook com teclado português (o caso típico desta disciplina); caso contrário, escolha o *layout* e a variante correspondentes ao seu teclado físico. Em seguida, no ecrã **Profile setup**, escolha um nome de utilizador e palavra-passe que consiga memorizar. No ecrã **SSH Setup**, marque **Install OpenSSH server**: é o que irá ficar à escuta na porta que acabou de redireccionar. No ecrã **Featured Server Snaps**, não seleccione nada; carregue em *Done*.

6. **Quando a instalação terminar**, desligue a VM em vez de reiniciar quando o *installer* o sugere. Retire a ISO (*Edit VM, Drives* na VM) e inicie a VM de novo. Deverá ver uma *prompt* de login na janela do UTM; pode deixá-la aí e passar para o Terminal do seu Mac:

**Se o *installer* só lhe ofereceu a opção "Reboot" (e não "Shutdown")**, poderá ver a VM a arrancar com uma linha vermelha `[FAILED] Failed unmounting /dev/cdrom`. O Ubuntu recém-instalado está a tentar desmontar a ISO, que continua anexada como CD virtual. A mensagem é inofensiva, mas a ISO tem mesmo de sair. Pare a VM à força a partir da barra de ferramentas do UTM, retire a ISO em *Edit VM, Drives* (remova a entrada de CD/DVD que aponta para `ubuntu-22.04.5-live-server-amd64.iso`), e inicie a VM de novo. A mensagem de falha desaparece e a *prompt* de login surge sem incidentes.

```
ssh -p 2222 <o-seu-utilizador>@localhost
```

Na *shell*, confirme:

```
uname -m           # x86_64
```

Opcionalmente adicione o mesmo bloco `~/.ssh/config` sugerido no Caminho 3 para que `ssh ssi-lab` baste.

Trate a VM como no Caminho 1 a partir daqui. Avance para [Secção 8: Verificar a configuração](#).

---

## Secção 8. Verificar a configuração

Abra a *shell* do ambiente que escolheu e execute:

```
uname -m           # deve imprimir x86_64
cat /etc/os-release | head -1 # deve mencionar Ubuntu 22.04
echo 'int main(){return 0;}' | cc -x c - -o /tmp/hello && /tmp/hello &&
echo "compile-and-run OK"
python3 --version   # Python 3.10.x
```

Se alguma linha falhar, consulte a [secção de resolução de problemas](#) abaixo antes de iniciar um trabalho.

As ferramentas específicas do trabalho ( `gcc` , `gdb` , `python3` , etc.) são instaladas a partir do enunciado de cada trabalho, não aqui.

---

## Secção 9. Resolução de problemas

### Docker (Caminhos 2b, 3b e 4)

**"Cannot connect to the Docker daemon":** o Docker Desktop não está a correr. Abra `/Applications/Docker.app` e espere que o ícone da baleia pare de animar.

**`docker compose` não é reconhecido:** a sua instalação do Docker é anterior ao Compose v2. Reinstale o Docker Desktop ou use `docker-compose` (com hífen) como substituto imediato.

**O *container* sai imediatamente:** verifique se a linha `command: sleep infinity` está presente no `docker-compose.yml` . Sem ela, o comando por defeito do Ubuntu termina e o *container* pára.

**GDB reporta `Operation not permitted` ao definir um *breakpoint*:** as linhas `cap_add` e `security_opt` no `docker-compose.yml` estão em falta. Reveja a Secção 4.4 e execute `docker compose up -d --force-recreate` .

**GDB reporta `Couldn't get registers: Input/output error` :** esta é a limitação Apple Silicon com Rosetta documentada na Secção 4.5. Use a alternativa de análise estática do enunciado, ou passe para o Caminho 5.

**Disco cheio:** a *imagem* de VM do Docker cresce ao longo do tempo. Recupere espaço com:

```
docker system prune --all --volumes
```

### VirtualBox (Caminho 3)

**A VM não arranca, "kernel driver not installed":** o macOS bloqueou a extensão de *kernel* do VirtualBox. Abra *Definições do Sistema, Privacidade e Segurança*, desça até ao fim e clique em *Permitir* ao lado de Oracle. Reinicie o Mac se solicitado.

**A rede não funciona:** nas *Definições, Rede* da VM, certifique-se de que o Adaptador 1 está ligado a *NAT*.

**Pastas partilhadas não aparecem:** instale as Guest Additions dentro da VM: `sudo apt install virtualbox-guest-utils` e reinicie.

## WSL2 (Caminho 2)

O Ubuntu nunca arranca após `wsl --install`: verifique se as funcionalidades Windows Plataforma de Máquina Virtual e Subsistema Windows para Linux estão activas (*Activar ou desactivar funcionalidades do Windows*). Reinicie.

As operações de ficheiro em `/mnt/c/...` são lentas: mantenha os seus ficheiros de trabalho em `~/...` (no Linux), não na unidade Windows montada.

## UTM (Caminho 5)

A instalação é extremamente lenta: é esperado, devido à emulação. Tire um *snapshot* após a instalação terminar para poder recuar rapidamente se necessário.

`sudo apt update` lento ou com *timeout*: na VM, prefira um *mirror* próximo: *Software e Atualizações, Descarregar de, Outro...* e escolha um servidor geograficamente próximo.

---

## Cartão de referência rápida

| Tarefa                                    | Comando                                                   |
|-------------------------------------------|-----------------------------------------------------------|
| Ver arquitectura                          | <code>uname -m</code>                                     |
| (Docker) Lançar o <i>container</i>        | <code>cd ~/ssi-lab &amp;&amp; docker compose up -d</code> |
| (Docker) Abrir uma <i>shell</i>           | <code>docker compose exec lab bash</code>                 |
| (Docker) Parar e remover <i>container</i> | <code>docker compose down</code>                          |
| (Todos os caminhos) Instalar ferramentas  | <code>sudo apt install gcc gdb python3 *</code>           |
| (Todos os caminhos) Ficheiros do trabalho | <code>work/ (macOS) ↔ /root/work (container)</code>       |

\* Omita o `sudo` quando já é *root* dentro do *container* Docker.

---

Se algo correr mal e não estiver coberto aqui, leve a mensagem de erro exacta para a aula ou para o fórum da disciplina, indicando o caminho que seguiu (1 a 5) para que a equipa docente possa ajudar rapidamente.

---

## Secção 10. Resolução de problemas alargada

As entradas abaixo cobrem problemas que surgem repetidamente nas sessões de laboratório e não estão completamente abordados na Secção 9.

## 10.1 SSH com ligação recusada ( `ssh -p 2222 ... localhost` devolve "Connection refused")

**Sintoma:** executa `ssh -p 2222 <utilizador>@localhost` a partir da *shell* do *host* e obtém `ssh: connect to host localhost port 2222: Connection refused`.

**Causas mais prováveis e correções:**

1. **O servidor OpenSSH não foi instalado no *guest*.** Durante o *installer* do Ubuntu Server, o ecrã **SSH Setup** pede-lhe que marque *Install OpenSSH server*. Se esse passo foi ignorado, o *guest* não tem servidor SSH. Corrija: arranque a VM, inicie sessão pela consola do VirtualBox ou UTM e instale manualmente:

```
sudo apt update && sudo apt install openssh-server
sudo systemctl enable --now ssh
```

2. **A regra de *port forward* não está activa.**
  - **VirtualBox:** em *Definições, Rede, Adaptador 1*, confirme que *Ligado a* está como **NAT** (não *Bridge* ou *Host-Only*). Confirme que existe uma regra de *port-forward* a mapear a porta 2222 do *host* para a porta 22 do *guest*. Se a VM foi iniciada antes de adicionar a regra, reinicie-a.
  - **UTM:** confirme que o *Network Mode* é **Emulated VLAN** (não *Shared Network*). Confirme que a entrada de *Port Forwarding* existe com *Guest Port 22* e *Host Port 2222*. Em caso de dúvida, pare a VM, verifique de novo *Settings, Network, Port Forwarding*, e reinicie.
3. **A VM ainda não terminou de arrancar.** O servidor SSH é um dos últimos serviços a iniciar. Aguarde até ver a *prompt* de login na janela do VirtualBox ou UTM antes de tentar ligar.
4. **Outro processo no seu *host* está a usar a porta 2222.** Execute `lsof -i :2222` (macOS/Linux) ou `netstat -ano | findstr :2222` (Windows) para verificar. Se algo estiver à escuta, escolha uma porta *host* diferente (por exemplo 2223) na regra de *port-forward* e ajuste o seu comando `ssh` em conformidade.

## 10.2 GDB em modo interativo falha dentro do *container* (Rosetta / *ptrace* em falta)

**Sintoma:** dentro de um *container* Docker num Mac Apple Silicon (Caminho 4), ao executar `gdb ./vuln` e depois `run`, obtém `Couldn't get registers: Input/output error` ou `ptrace: Operation not permitted`.

**Causa:** o Rosetta 2 não consegue fazer `ptrace` sobre os processos x86-64 que está a traduzir. Esta é uma limitação documentada do caminho Docker com Rosetta e não pode ser contornada sem mudar de ambiente.

**Opções de correção (escolha uma):**

- **Use a alternativa de análise estática** fornecida no Exercício 2 do enunciado da Semana 11. Produz as mesmas quatro respostas (endereço do *buffer*, *offset* do *return address*, endereço de `secret_function`) sem se ligar a um processo em execução.
- **Mude para o Caminho 5 (UTM).** O UTM executa um *kernel* x86-64 totalmente emulado; o GDB funciona exactamente como descrito em *hardware* nativo. A contrapartida é o tempo de configuração único de 60 a 90 minutos.
- **Se estiver no Caminho 3 ou 3b (Mac Intel ou *host* x86-64) e vir `ptrace: Operation not permitted`:** as linhas `cap_add: SYS_PTRACE` e `security_opt: seccomp=unconfined` estão em falta no `docker-compose.yml`. Reveja a Secção 3b.2 ou 4.4 e execute `docker compose up -d --force-recreate` para recriar o *container* com as capacidades correctas.

## 10.3 Problemas de teclado (teclado português no Ubuntu Server em VM: @, |, ~ no sítio errado)

**Sintoma:** ao escrever na janela de consola do VirtualBox ou UTM (ou numa sessão SSH configurada com o *locale* errado), os caracteres `@`, `|`, `~`, `{`, `}`, `[`, `]` e `\` não aparecem onde esperado.

**Causa:** o *installer* usou um *layout* de teclado que não corresponde ao teclado físico.

**Correcção durante a instalação:** no ecrã **Keyboard configuration** do *installer* do Ubuntu, defina:

- **Layout:** Portuguese
- **Variant:** Portuguese - Portuguese (Macintosh) (para teclados MacBook) ou Portuguese (para teclados PT normais).

**Correcção após a instalação (se o *layout* errado foi seleccionado):** dentro do *guest* em execução, execute:

```
sudo dpkg-reconfigure keyboard-configuration
```

Siga as instruções: seleccione o modelo do teclado, depois escolha *Portuguese* como *layout* e a variante adequada. Aplique a alteração com:

```
sudo setupcon
```

**Alternativa para sessões SSH:** o *layout* de teclado só importa quando escreve directamente numa janela de consola. Uma vez ligado via SSH a partir do terminal do *host*, aplica-se o *layout* do *host*, não o do *guest*. Se achar a consola difícil de usar, configure o SSH (Secção 3.5 ou passo 6 do Caminho 5) cedo para poder parar de escrever na janela de consola.

## 10.4 Confusão com a ordem das colunas de *port forwarding* no UTM

**Sintoma:** adiciona uma regra de *port forwarding* no UTM mas o SSH continua a não ligar, ou a regra parece errada depois de a introduzir.

**Contexto:** a tabela de *Port Forwarding* do UTM usa uma ordem de colunas diferente da do VirtualBox. Alunos que completaram o trabalho de Segurança Web no VirtualBox por vezes introduzem as portas na ordem errada.

**Ordem correcta das colunas no UTM:**

| Protocol | Guest Address  | Guest Port | Host Address   | Host Port |
|----------|----------------|------------|----------------|-----------|
| TCP      | (deixar vazio) | 22         | (deixar vazio) | 2222      |

A diferença principal em relação ao VirtualBox: no UTM as colunas do **Guest** vêm antes das do **Host** (Guest à esquerda, Host à direita). No diálogo de *Port Forwarding* do VirtualBox, a ordem é Nome, Protocolo, IP Host, Porta Host, IP Guest, Porta Guest (Host à esquerda).

Se já introduziu a regra ao contrário (*Guest Port 2222, Host Port 22*), apague-a e adicione-a novamente com os valores trocados.

## 10.5 Instalação de UTM muito lenta: porque demora tanto e o que fazer

**Duração esperada:** a instalação completa do Ubuntu 22.04 Server sob UTM num Mac Apple Silicon demora **60 a 90 minutos ou mais** numa primeira instalação. Alguns alunos relataram mais de duas horas em máquinas M1 mais antigas com armazenamento mais lento.

**Porque é lento:** o UTM emula um CPU x86-64 inteiramente em *software*. O *installer* do Ubuntu não se limita a copiar ficheiros: executa o gestor de pacotes APT, que realiza resolução de dependências, desempacotamento e *scripts* pós-instalação para cada pacote. O Ubuntu 22.04 LTS tem vindo a acumular actualizações de segurança desde a sua publicação em 2022, e o *installer* aplica-as todas durante a primeira execução. O multiplicador de emulação (aproximadamente 5 a 10 vezes mais lento que ARM64 nativo) aplica-se a cada *script* de pacote, o que soma rapidamente.

**O que fazer durante a espera:**

- Deixe a VM em execução e trabalhe noutra coisa. O *installer* não requer atenção depois de pressionar *Done* no ecrã *Featured Server Snaps*.
- Não feche o UTM à força nem ponha o Mac em *sleep* enquanto o *installer* está em execução; isso pode corromper o disco virtual.



- Se o *installer* parecer bloqueado durante mais de 20 minutos sem actividade de disco (pode observar o ícone de disco da VM no UTM a piscar), experimente pressionar Enter uma vez; por vezes o *installer* está à espera de resposta a uma *prompt* fora do ecrã.

**Após a conclusão da instalação:** tire imediatamente um *snapshot* (*menu VM, Snapshot, Save*) para poder voltar atrás rapidamente se algo correr mal mais tarde, sem repetir a instalação completa.

**Da próxima vez:** com o *snapshot* no lugar, iniciar a VM a partir de um estado desligado demora menos de 60 segundos, mesmo sob emulação.

## 10.6 Erros de compilação: `gcc: command not found`, `python3` em falta

**Sintoma:** dentro da *shell* x86-64, ao executar `gcc` ou `python3`, obtem `command not found`.

**Causa:** estas ferramentas não estão pré-instaladas na imagem Docker base do Ubuntu 22.04 nem numa VM Ubuntu Server recém-instalada. O guia de configuração e o enunciado pedem que as instale, mas este passo é fácil de saltar.

**Correção (*container Docker*, já é *root*):**

```
apt update && apt install -y gcc gdb python3
```

**Correção (VM ou WSL2, utilizador regular):**

```
sudo apt update && sudo apt install gcc gdb python3
```

A instalação demora alguns minutos. Se `apt update` falhar com um erro de rede, verifique que o adaptador de rede da VM está definido como NAT (VirtualBox) ou Emulated VLAN (UTM) e que o *host* tem acesso à internet.

**Se `cc` existe mas `gcc` não:** `cc` é um alias POSIX que pode apontar para `gcc` ou `clang`. Os enunciados usam `gcc` explicitamente para controlar as *flags* de mitigação. Instale o pacote `gcc` como indicado acima.

## 10.7 A mensagem do *stack canary* parece uma falha do sistema

**Sintoma:** quando recompila `vuln.c` com o *stack canary* activo (Exercício 4, primeira linha da tabela) e executa o *payload* de *overflow*, o terminal imprime:

```
*** stack smashing detected ***: terminated
Aborted (core dumped)
```

Muitos alunos interpretam `Aborted (core dumped)` como uma falha do sistema operativo ou da ferramenta e reportam que "algo correu mal".

**Explicação:** este é o comportamento correcto e esperado da mitigação *stack canary*. O que aconteceu:

1. O *payload* ainda sobrescreveu o `buffer` e atingiu o valor *canary* colocado pelo compilador entre o *buffer* e o *return address* guardado.
2. No retorno de `process_input`, o *runtime* comparou o valor do *canary* com o valor esperado. Os valores diferiam, confirmando a corrupção.
3. O *runtime* chamou `__stack_chk_fail`, que imprimiu o diagnóstico e enviou `SIGABRT` ao processo, terminando-o imediatamente sem transferir o controlo para `secret_function`.

Este é o *stack canary* a funcionar exactamente como foi concebido. A palavra "falha" é uma formulação enganosa: o programa detectou um ataque e terminou de forma controlada, impedindo o atacante de atingir execução de código arbitrária. Documente isto no seu relatório e explique porque uma terminação controlada é preferível a comportamento indefinido.

**Nota:** `core dumped` significa que o SO escreveu um *snapshot* de memória para disco (se os *core dumps* estiverem activos). Na maioria das configurações o ficheiro *core* é suprimido por defeito. Não precisa do ficheiro *core* para este exercício.

## 10.8 "Normal programme termination" impresso antes de ACCESS GRANTED (confusão de *timing* ou saída)

**Sintoma:** após um *overflow* aparentemente bem-sucedido no Exercício 3, a saída do terminal mostra tanto `[*] Normal programme termination.` como `[!] ACCESS GRANTED: you reached the secret function!`, nessa ordem, ou a mensagem `ACCESS GRANTED` não aparece de todo apesar de o *payload* parecer correcto.

**Explicação:** existem dois cenários distintos:

1. **O *payload* está ligeiramente errado (mais frequente):** o *return address* foi sobrescrito, mas não com o endereço exacto de `secret_function`. O programa voltou para algum local que por acaso executou algumas instruções antes de falhar com uma falha de segmentação, o que pode ou não ter executado o `printf` em `main`. Verifique de novo o *offset* (deve ser 72 bytes em Ubuntu 22.04 com GCC 11) e a codificação *little-endian* do endereço de `secret_function`.
2. **Ordenação do *output* (fluxo com *buffer*):** em alguns emuladores de terminal, o *output* de `printf("[*] Normal programme termination.\n")` em `main` e o *output* de `printf("[!] ACCESS GRANTED ...")` em `secret_function` podem entrelaçar-se se o processo estiver a escrever para um *stream* com *buffer*. Na prática isto é raro porque `secret_function` chama `exit(0)` antes de regressar a `main`. Se vir ambas as

mensagens, quase de certeza que se trata do cenário 1 (*overflow* parcial para o local errado).

**O que procurar num *exploit* correcto:** o processo termina via `exit(0)` dentro de `secret_function`, pelo que `main` nunca chega à linha `Normal programme termination`. Um *exploit* bem-sucedido produz uma saída terminada com o bloco `ACCESS GRANTED` e sem a linha `Normal programme termination`.